07 — Desbalanceo de Clases

Pipeline CAD-TBX11K | Diagnóstico Asistido por Computador en Tuberculosis

Este notebook selecciona la estrategia de desbalanceo usando **todas las imágenes** (8274) y **validación cruzada**. No hay hold-out ni test separado: la estimación es la media de la CV.

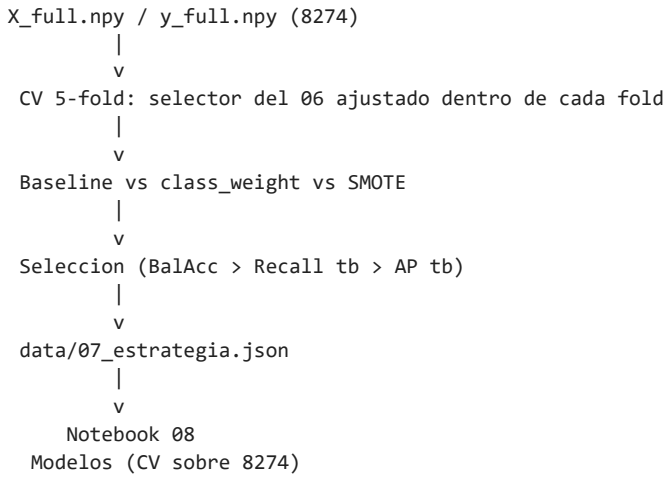
Estrategias

Estrategia	Familia	Descripción
Baseline	—	`LogisticRegression` sin corrección
`class_weight='balanced'`	Modelo	Pondera el costo por frecuencia inversa
SMOTE	Datos	Genera muestras sintéticas de `tb` (solo en train de cada fold)

Protocolo

1. Cargar `X_full.npy` / `y_full.npy` (8274 imágenes).
2. **CV 5-fold estratificada**.
3. En cada fold, el **selector de features del 06 se ajusta solo con el train** y se aplica a train/validación.
4. Comparar las tres estrategias con el mismo clasificador y escalado.
5. Elegir por **balanced accuracy > recall de `tb` > AP de `tb`**.

Flujo



0. Configuración del entorno

In [1]:
```
# Se configura el path del proyecto y se importan las librerías necesarias.
# Referencias:
#   imbalanced-learn: https://imbalanced-learn.org/stable/
#   StratifiedKFold: https://scikit-
learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html

import sys, os, warnings, json, time
warnings.filterwarnings('ignore')
```

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    balanced_accuracy_score, f1_score, recall_score, average_precision_score,
)

from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

PROJECT_ROOT = os.path.abspath(os.path.join(os.getcwd(), '..'))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

from src.selection import build_selector
from src.visualization import set_publication_style

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

CLASS_NAMES = ['health', 'sick', 'tb']
N_FOLDS     = 5
DATA_DIR    = Path('..') / 'data'

set_publication_style()
print('Entorno configurado correctamente.')
```

```
Entorno configurado correctamente.
```

1. Carga de las 8274 imágenes y de la selección del 06

In [2]:

```
# Se cargan las 8274 imágenes (sin split) y la selección method/N del notebook 06.
# Referencia: https://numpy.org/doc/stable/reference/generated/numpy.load.html

for archivo in ['X_full.npy', 'y_full.npy', '06_seleccion.json']:
    if not (DATA_DIR / archivo).exists():
        raise FileNotFoundError(
            f'No se encontró {archivo}. Ejecuta primero el notebook 06.'
        )

X = np.load(DATA_DIR / 'X_full.npy')
y = np.load(DATA_DIR / 'y_full.npy')

with open(DATA_DIR / '06_seleccion.json', encoding='utf-8') as f:
    seleccion = json.load(f)

METHOD_SEL = seleccion['metodo']
N_SEL = int(seleccion['n_features'])

print(f'X : {X.shape} ({X.dtype})')
print(f'y : {y.shape}')
print()
print('Distribución:')
for clase, conteo in zip(*np.unique(y, return_counts=True)):
    print(f'  {CLASS_NAMES[clase]:6s} (label={clase}): {conteo} muestras')
print()
print(f'Selector del 06: {METHOD_SEL}, N={N_SEL}')
```

```
X : (8256, 3300) (float32)
y : (8256,)

Distribución:
  health (label=0): 3800 muestras
  sick   (label=1): 3656 muestras
  tb     (label=2): 800 muestras

Selector del 06: MI, N=2000
```

2. Selector y estrategias

In [3]:
```python
# El selector se ajusta dentro de cada fold. Las tres estrategias usan el mismo
# clasificador y el mismo escalado.
# Referencia: https://imbalanced-learn.org/stable/references/generated/imblearn.pipeline.Pipeline.html

selector_base = build_selector(METHOD_SEL, N_SEL, random_state=RANDOM_STATE)


def construir_estrategias():
    """Devuelve nombre -> pipeline sin ajustar."""
    return {
        'Baseline': Pipeline([
            ('scaler', StandardScaler()),
            ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
        ]),
        'class_weight': Pipeline([
            ('scaler', StandardScaler()),
            ('clf', LogisticRegression(
                class_weight='balanced', max_iter=1000, random_state=RANDOM_STATE,
            )),
        ]),
        'SMOTE': ImbPipeline([
            ('scaler', StandardScaler()),
            ('smote', SMOTE(random_state=RANDOM_STATE)),
            ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)),
        ]),
    }


ESTRATEGIAS = construir_estrategias()
print('Selector:', type(selector_base).__name__)
print('Estrategias:', list(ESTRATEGIAS))
```

```
Selector: SelectKBest
Estrategias: ['Baseline', 'class_weight', 'SMOTE']
```

3. Validación cruzada sobre las 8274

In [4]:
```python
# CV 5-fold sobre las 8274. En cada fold, el selector se ajusta SOLO con el train
# del fold y transforma train/validación. No hay hold-out.
# Referencia: https://scikit-learn.org/stable/modules/cross_validation.html

cv = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_STATE)

filas_resumen = []
filas_fold = []

for nombre in ESTRATEGIAS:
    t0 = time.time()
    metricas_fold = []

    for fold, (tr_idx, va_idx) in enumerate(cv.split(X, y), start=1):
        # Selector ajustado solo con el train del fold.
        selector = clone(selector_base).fit(X[tr_idx], y[tr_idx])
        X_tr = selector.transform(X[tr_idx])
        X_va = selector.transform(X[va_idx])
        y_tr, y_va = y[tr_idx], y[va_idx]

        modelo = clone(ESTRATEGIAS[nombre])
```

```
        modelo.fit(X_tr, y_tr)
        y_pred = modelo.predict(X_va)
        y_proba = modelo.predict_proba(X_va)

        fila = {
            'Estrategia': nombre,
            'Fold': fold,
            'Balanced Acc': balanced_accuracy_score(y_va, y_pred),
            'F1 macro': f1_score(y_va, y_pred, average='macro'),
            'Recall tb': recall_score(y_va, y_pred, labels=[2], average=None)[0],
            'AP tb': average_precision_score((y_va == 2).astype(int), y_proba[:, 2]),
        }
        metricas_fold.append(fila)
        filas_fold.append(fila)

    df_fold = pd.DataFrame(metricas_fold)
    resumen = {
        'Estrategia': nombre,
        'BalAcc_media': df_fold['Balanced Acc'].mean(),
        'BalAcc_std': df_fold['Balanced Acc'].std(),
        'F1_media': df_fold['F1 macro'].mean(),
        'F1_std': df_fold['F1 macro'].std(),
        'RecallTB_media': df_fold['Recall tb'].mean(),
        'RecallTB_std': df_fold['Recall tb'].std(),
        'AP_media': df_fold['AP tb'].mean(),
        'AP_std': df_fold['AP tb'].std(),
        'Tiempo_s': time.time() - t0,
    }
    filas_resumen.append(resumen)
    print(f'{nombre:14s}  BalAcc={resumen["BalAcc_media"]:.4f} '
          f'+/- {resumen["BalAcc_std"]:.4f}  F1={resumen["F1_media"]:.4f}  '
          f'AP(tb)={resumen["AP_media"]:.4f}  ({resumen["Tiempo_s"]:.1f}s)')

resultados_cv = pd.DataFrame(filas_resumen).sort_values(
    ['BalAcc_media', 'RecallTB_media', 'AP_media'], ascending=False,
).reset_index(drop=True)
resultados_fold = pd.DataFrame(filas_fold)

print()
print('Resultados de validación cruzada:')
print(resultados_cv.round(4).to_string(index=False))

resultados_cv.to_csv(DATA_DIR / '07_cv_resultados.csv', index=False)
resultados_fold.to_csv(DATA_DIR / '07_cv_por_fold.csv', index=False)
```

```
Baseline       BalAcc=0.7407 +/- 0.0135  F1=0.7406  AP(tb)=0.4292  (374.8s)
class_weight   BalAcc=0.7420 +/- 0.0118  F1=0.7349  AP(tb)=0.4334  (387.9s)
SMOTE          BalAcc=0.7396 +/- 0.0070  F1=0.7338  AP(tb)=0.4275  (411.8s)

Resultados de validación cruzada:
  Estrategia  BalAcc_media  BalAcc_std  F1_media  F1_std  RecallTB_media  RecallTB_std  AP_media  AP_std
Tiempo_s
class_weight        0.7420      0.0118    0.7349  0.0097          0.4738        0.0349    0.4334  0.0242
387.8602
    Baseline        0.7407      0.0135    0.7406  0.0119          0.4438        0.0356    0.4292  0.0289
374.7639
       SMOTE        0.7396      0.0070    0.7338  0.0057          0.4662        0.0244    0.4275  0.0155
411.8335
```

4. Selección de la estrategia

In [5]:

```
# Selección por balanced accuracy; empate -> recall de tb -> AP de tb.
# La decisión se persiste para el notebook 08.

MEJOR_ESTRATEGIA = resultados_cv.loc[0, 'Estrategia']

print(f'Estrategia seleccionada por CV: {MEJOR_ESTRATEGIA}')
print()
for _, fila in resultados_cv.iterrows():
    print(f'  {fila["Estrategia"]:14s}  BalAcc={fila["BalAcc_media"]:.4f} '
          f'+/- {fila["BalAcc_std"]:.4f}  RecallTB={fila["RecallTB_media"]:.4f}  '
          f'AP(tb)={fila["AP_media"]:.4f}')
```

```python
decision = {
    'estrategia': MEJOR_ESTRATEGIA,
    'criterio': ['balanced_accuracy', 'recall_tb', 'average_precision_tb'],
    'n_folds': N_FOLDS,
    'n_imagenes': int(len(y)),
    'resultados_cv': resultados_cv.round(6).to_dict(orient='records'),
}
with open(DATA_DIR / '07_estrategia.json', 'w', encoding='utf-8') as f:
    json.dump(decision, f, ensure_ascii=False, indent=2)

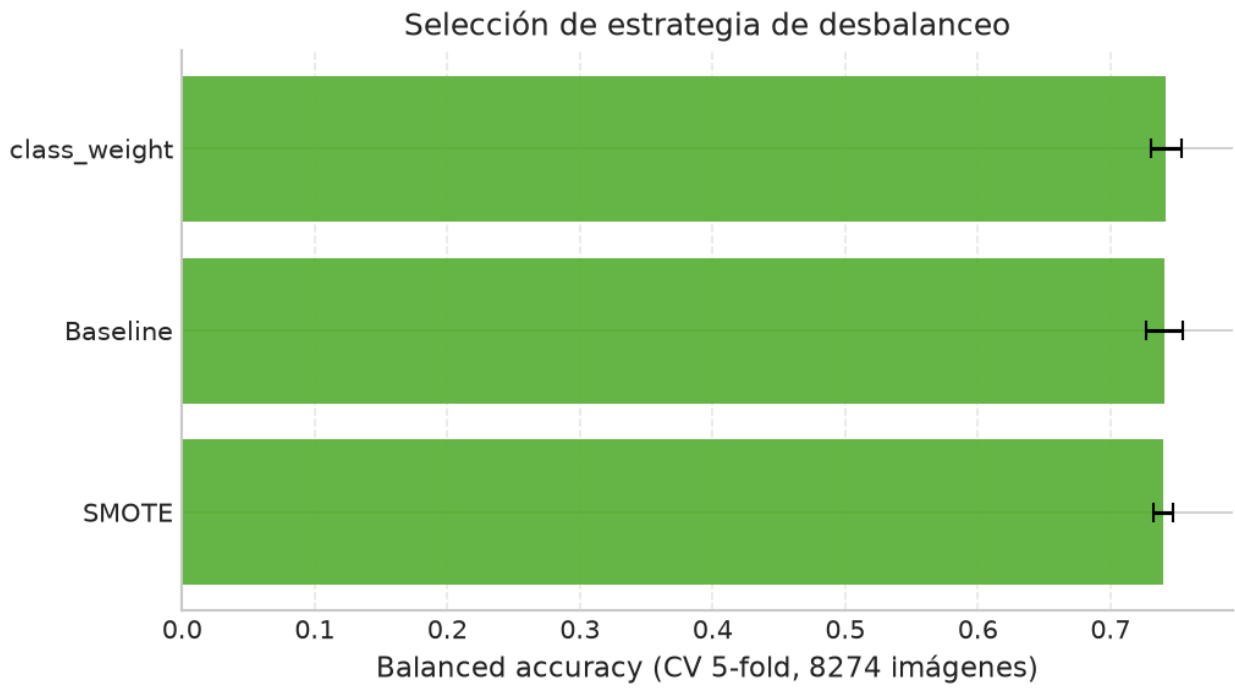
print()
print(f'Decisión guardada en: {(DATA_DIR / "07_estrategia.json").resolve()}')

fig, ax = plt.subplots(figsize=(7, 4))
orden = resultados_cv.sort_values('BalAcc_media')
ax.barh(orden['Estrategia'], orden['BalAcc_media'], xerr=orden['BalAcc_std'],
        capsize=4, color='#4DAC26', alpha=0.85)
ax.set_xlabel('Balanced accuracy (CV 5-fold, 8274 imágenes)', fontsize=11)
ax.set_title('Selección de estrategia de desbalanceo', fontsize=12)
ax.grid(True, axis='x', linestyle='--', alpha=0.4)
fig.tight_layout()
plt.show()
```

```
Estrategia seleccionada por CV: class_weight

  class_weight    BalAcc=0.7420 +/- 0.0118  RecallTB=0.4738  AP(tb)=0.4334
  Baseline        BalAcc=0.7407 +/- 0.0135  RecallTB=0.4437  AP(tb)=0.4292
  SMOTE           BalAcc=0.7396 +/- 0.0070  RecallTB=0.4662  AP(tb)=0.4275

Decisión guardada en: C:\Users\estra\Desktop\Tuberculosis\cad-tbx11k\data\07_estrategia.json
```



Conclusiones

Se compararon las tres estrategias con **CV 5-fold sobre las 8274 imágenes**. En cada fold, el selector de características se ajustó solo con el train y transformó train/validación.

La estrategia elegida se guarda en `data/07_estrategia.json` y los resultados en `data/07_cv_resultados.csv` y `data/07_cv_por_fold.csv`.

No hay hold-out: la estimación es la media de la CV sobre todas las imágenes. El notebook 08 usará la misma lógica (selector dentro de cada fold) para entrenar y evaluar los modelos.

Siguiente etapa: `08_Modelos_ML.ipynb`